Time & Space Complexity
Big O
TLE → Time Limit Exceeded

} TC 2 lecture

Today → How to find # iterations in a code. ✓

Basic Maths

1) Sum of first N natural numbers $= \frac{N(N+1)}{2}$ Eg → N = 4
$1+2+3+4 = \underline{10}$

2) count of numbers in a range L to R. → $\underline{R-L+1}$
[3 8] → {3, 4, 5, 6, 7, 8} $\underline{6}$ $8-3+1 = \underline{6}$

1 2 3 4 5 6 7 8
L-1 L

$R-(L-1) = \underline{R-L+1}$ ✓

3) count of times N is divided by 2 to reach 1 → $\underline{\log_2(N)}$
↳ (Refer last class) N = 8 $\frac{8}{2} \to \frac{4}{2} \to \frac{2}{2} \to 1$
$\underline{3 \text{ steps}}$

Arithmetic Progression

Eg → 4 7 10 13 16 19 . . .

$7-4=3$ $13-10=3$
$10-7=3$ $16-13=3$

first term = a
common difference = d

a (a+d) (a+2d) (a+3d)
4 (4+3) (4+2*3) (4+3*3) . . .

$S_n = a + (a+d) + (a+2d) + \dots (a+(n-1)d)$
$S_n = (a+(n-1)d) + (a+(n-2)d) + (a+(n-3)d) + \dots a$

$$2S_n = (2a+(n-1)d) + (2a+(n-1)d) + (2a+(n-1)d) + \dots (2a+(n-1)d)$$

$$\Rightarrow 2S_n = n*(2a+(n-1)d) \Rightarrow \boxed{S_n = \frac{n}{2}(2a+(n-1)d)}$$

Geometric Progression

Eg → 2 6 18 54 . . .

$2*3$ $6*3$ $18*3$

$6/2 = 3$ $18/6 = 3$ $54/18 = 3$

first term = a

common ratio = r

$$S_n = a + a*r + a*r^2 + \dots + a*r^{n-1}$$

$$S_n*r = a*r + a*r^2 + \dots a*r^{n-1} + a*r^n$$

$$-S_n = a + a*r + a*r^2 + \dots + a*r^{n-1}$$

$$S_n*r - S_n = -a + a*r^n$$

$$\Rightarrow S_n(r-1) = a(r^n-1) \Rightarrow \boxed{S_n = \frac{a(r^n-1)}{(r-1)}}$$

Q → $\log_a(a^x) = \underline{x}$

$\log_2(2^3) = k \rightarrow 2^k = 2^3 \Rightarrow k = \underline{3}$

Find # iterations

1)
```
for (int i = 1 ; i <= N ; i++) {
        print (i)
}
```

$i \rightarrow [1 \quad N] \rightarrow \underline{N}$ iterations

$TC = \underline{O(N)}$

2)
```
for (int i = 1; i <= N; i++) {
    if (i % 2 == 0)
        print(i)
}
```
$i \rightarrow [1 \quad N] \rightarrow$ N iterations

```
for (int i = 1; i <= M; i++) {
    if (i % 2 == 0)
        print(i)
}
```
$i \rightarrow [1 \text{ to } M] \rightarrow$ M iterations

total = (N+M) iterations

TC = $O(N+M)$ or $O(\max(N, M))$

3)
```
int s = 0
for (int i = 1; i <= N; i = i + 2) {
    s = s + i
}
```
$9/2 \rightarrow 4$

$\#\text{iterations} = \frac{N+1}{2} = N * \frac{1}{2} + \frac{1}{2}$

ceil(N/2)

N = 5 $\quad i \rightarrow \{1, 3, 5\} \rightarrow 3$

N = 6 $\quad i \rightarrow \{1, 3, 5\} \rightarrow 3$

N = 7 $\quad i \rightarrow \{1, 3, 5, 7\} \rightarrow 4$

N = 8 $\quad i \rightarrow \{1, 3, 5, 7\} \rightarrow 4$

TC = $O(N)$

4)
```
int s = 0
for (int i = 0; i <= 100; i++) {
    s = s + i + i*i
}
return s
```
$i \rightarrow [0 \quad 100] \rightarrow 100 - 0 + 1 = 101$

$\#\text{iterations} = 101$

TC = $O(1)$

5)
```
int s = 0
for (int i = 1; i*i <= N; i++) {
    s = s + i
}
```
$i^2 <= N \Rightarrow i <= \sqrt{N}$

$i \rightarrow 1 \text{ to } \sqrt{N}$

$\#\text{iterations} = \sqrt{N}$

TC = $O(\sqrt{N})$

6) int i = N // N > 0

while (i > 1) {

i /= 2

}

$\log_x(1) = 0 \longrightarrow x^0 = 1$

iterations = $\log_2(N)$

TC = $O(\log(N))$

N = 10 i → 10 → 5 → 2 → 1

N = 8 i → 8 → 4 → 2 → 1

7) int s = 0

for (int i = 0 ; i < N ; i = i * 2) {

s = s + i

} return s

iterations = ∞ ✓

i = 0 → 0 * 2 = 0 → 0 * 2 = 0 . . . .

8) int s = 0

for (int i = 1 ; i < N ; i = i * 2) {

s = s + i

} return s

$\log_x(y) = \dfrac{\log_z(y)}{\log_z(x)}$

iterations = ceil $(\log_2(N))$

i → 1 → 2 → 4 → 8 . . . N

TC = $O(\log(N))$

9) for (int i = 1 ; i <= 10 ; i++) {

for (int j = 1 ; j <= N ; j++) {

print (i+j)

}

}

i	j	# iterations
1	[1 N]	N
2	[1 N]	N
⋮		⋮
10	[1 N]	N

TC = $O(N)$

$10 * N$ #iterations $= 10 * N$

10)
```
for (int i = 1; i <= N; i++) {
    for (int j = 1; j <= N; j++) {
        print (i+j)
    }
}
```

i	j	# iterations
1	[1 N]	N
2	[1 N]	N
⋮		
N	[1 N]	N
		N * N

TC = $O(N^2)$

#iterations $= N^2$

10:30 PM

11)
```
for (int i = 0; i < N; i++) {
    for (int j = 0; j <= i ; j++) {
        print (i+j)
    }
}
```

i	j	# iterations
0	[0 0]	1
1	[0 1]	2
2	[0 2]	3
3	[0 3]	4
⋮		
(N-1)	[0 (N-1)]	N

#iterations $= \frac{N(N+1)}{2} = \frac{N^2}{2} + \frac{N}{2}$

TC = $O(N^2)$

$1 + 2 + 3 + 4 + \ldots + N = \frac{N(N+1)}{2}$

12)
```
for (int i = 1; i <= N; i++) {
    for (int j = 1; j <= N; j = j*2) {
        print (i+j)
    }
}
```

→ $\log_2(N)$

i	j	# iterations
1	1 → N	$\log_2(N)$
2	1 → N	$\log_2(N)$
3	1 → N	$\log_2(N)$
⋮		⋮
N	1 → N	$\log_2(N)$
		$N * \log_2(N)$

iterations = $N * \log_2(N)$

TC = $O(N \log(N))$

13)
```
for (int i = 1; i <= 2^N; i++) {
    print (i)
}
```

$i \rightarrow [1 \quad 2^N] \rightarrow 2^N$

TC = $O(2^N)$

14)
```
for (int i = 1; i <= N; i++) {
    for (int j = 1; j <= 2^i; j++) {
        print (i+j)
    }
}
```

i	j	# iterations
1	$[1 \quad 2^1]$	2^1
2	$[1 \quad 2^2]$	2^2
3	$[1 \quad 2^3]$	2^3
⋮	⋮	
N	$[1 \quad 2^N]$	2^N
		$2^1 + 2^2 + 2^3 + \dots 2^N$

$2^{N+1} = 2 * 2^N$

$$\frac{a(r^{n}-1)}{(r-1)} \qquad \frac{2^{1}(2^{N}-1)}{(2-1)} = \underline{2(2^{N}-1)} \qquad TC = \underline{O(2^{N})}$$

15)
```
for (int i = N; i > 0; i = i/2) {
    for (int j = 1; j <= i; j++) {
        print (i+j)
    }
}
```

i	j	# iterations
N	[1 N]	N
N/2	[1 N/2]	N/2
N/4	[1 N/4]	N/4
⋮		
1	[1 1]	1

$$N + \frac{N}{2} + \frac{N}{4} + \dots + 1 \qquad \boxed{\#\text{terms} = \log_2(N)}$$

$$\frac{N((1/2)^{\log_2(N)}-1)}{(1/2)-1} = \frac{N(1-(1/2^{\log_2(N)}))}{1-(1/2)} = \frac{N(1-1/N)}{(1/2)} = 2N\left(1-\frac{1}{N}\right)$$

$$\Rightarrow \frac{2N(N-1)}{N} = \underline{2(N-1)} \checkmark$$

$$\boxed{\frac{x-y}{a-b} = \frac{y-x}{b-a}}$$

$$TC = \underline{O(N)}$$

$$2^{\log_2(N)} = x \longrightarrow \log_2(x) = \log_2(N) \Rightarrow \underline{x = N}$$

For $N > 100$, $\log_2(100) = 6$

$\sqrt{100} = 10$

2^{100}

$$\log_2(N) < \sqrt{N} < N < N^2$$

$$\log_2(N) < \sqrt{N} < N < N*\log(N) < N^2 < 2^N < N!$$

$1*2*3*4*...*N$

Time complexity → O(# iterations) ✓

↓

Big O → rate of growth of function

$f(N) = 2N^2 - 3N$ → quadratic

$O(\cancel{2}N^2 - \cancel{3N}) \rightarrow O(N^2)$

constant lower order terms

1) $f(N) = N^3 + 10N - \log_2(N) \rightarrow O(N^3)$

2) $f(N) = N! + 3N^2 + N \rightarrow O(N!)$

3) $f(N) = 4N\log(N) + 3N\sqrt{N} + 100 \rightarrow O(N\sqrt{N})$